

Tool: Reference Architecture Library

Apa: kumpulan **pola arsitektur teruji** buat stack ini (React + Zustand + Chakra v2 + Fauxbase) yang tim bisa **reuse** — biar gak reinvent tiap fitur. **Kapan dipakai:** SOP-SG-05, sebelum desain fitur baru. Cek sini dulu (Reuse > Rebuild). Pola di sini = **guardrail self-serve** (Type-2) — tim jalan sendiri tanpa nunggu approval. **Framework:** TOGAF ADM C (Information Systems), SWEBOK (Software Design), mandate Tata F-1/F-2/Data SACRED.

*Pola di sini **proven** di prototype aktif. Kalau butuh nyimpang, itu keputusan baru → cek reversibility-matrix.*

Pola 1 — Data Layer: Entity + Service + Hooks

Kapan dipakai: semua akses data. Default, gak ada pengecualian buat prototype. **Struktur:**

```
entity/Gift.js      → class Gift extends Entity { @field name; @field claimedBy; ... }
service/GiftService → class GiftService extends Service { entity = Gift; pollClaims(){...} }
client.js           → createClient({ driver: { type: 'local', persist: 'memory' } })
komponen React      → useList(GiftService) / useGet / useMutation (dari fauxbase-react)
```

Guardrail:

- Custom logic (filter, agregasi, polling) **masuk Service method**, jangan di komponen (F-1 backend almighty).
- Komponen cuma konsumsi hooks — **jangan akses driver langsung**.
- Seed data disiapkan di init buat prototype.

JANGAN dipakai kalau: — (gak ada; ini default wajib). Mock data manual / Express backend = **haram** buat prototype.

Pola 2 — Config-Driven Feature Toggle

Kapan dipakai: fitur yang user mau bisa **on/off + editable** (Tata sering minta ini — hampir semua field). Hindari hardcoded. **Struktur:**

```
entity/InvitationConfig → @field sections (array {key, enabled, label, order, editable})
komponen                 → render dari config.sections.filter(s => s.enabled).sort(order)
dashboard                → user toggle enabled + edit label → useMutation simpan ke config
```

Guardrail:

- **Jangan hardcoded** daftar section/field di komponen — baca dari config entity.
- Tambah section baru = tambah row config, **bukan** edit komponen (open/closed).
- Default value aman kalau config kosong (graceful).

JANGAN dipakai kalau: field benar-bener fixed selamanya & gak akan pernah user-editable (rare — default tetap config-driven karena Tata maunya gitu).

Pola 3 — Merge-Not-Overwrite (Data SACRED)

Kapan dipakai: tiap update data yang udah ada. Mandate Data SACRED (R-DATA-MERGE).

Struktur:

```
update: const prev = await GiftService.get(id);
        await GiftService.update(id, { ...prev, ...patch, history: [...prev.history, {ts, change}]
delete: soft-delete → @field deletedAt; query default filter deletedAt == null
```

Guardrail:

- **Never overwrite** seluruh record — selalu spread `prev` dulu, merge patch.
- **No hard delete** — soft-delete via `deletedAt`, data tetap ada.
- Tiap perubahan **append ke history/audit trail** (F-1, audit-comply).
- **No auto-overwrite silent** — tiap merge ada logging eksplisit.

JANGAN dipakai kalau: — (gak ada; merge + soft-delete = default wajib semua entity).

Reuse pola ini = konsistensi + cepat + comply mandate. Nyimpang dari pola = keputusan arsitektur baru → klasifikasi reversibility + (kalau Type-1) ADR.